

COURSEWORK REPORT

Drive Circuit

A Route-Based Graphics using OpenGL

Jaygovindhan Neelananarayanan

Student ID: 240038757

Module: INM376 Computer Graphics

Instructor: Dr. Eddie Edwards

City, University of London 2026

1. Project Overview

Drive Circuit is a single-player graphics game written in C++ with OpenGL 4.0 and GLSL shaders. The player drives a red sports car around an F1-style circuit that I built using a Catmull-Rom spline. Rather than a simple oval, the track has 16 control points that produce a layout with two hairpin corners, an S-curve section, a chicane, and long sweeping straights — giving it a genuine circuit feel.

The game runs in three visual modes. In Day mode, the scene is lit by directional sunlight. Pressing L switches to Night mode, which turns off the sun and activates two spotlights — a white headlight that follows the car and an orange flood at the far side of the circuit. Pressing K activates Storm mode on top of Night, adding heavy fog, a narrower headlight cone, animated rain streaks across the screen, and unpredictable lightning flashes that light the whole scene.

The scene is populated with 8 trees, 4 statues, 4 stone structures, and 40 animated diamond markers along both track edges. The central diamond gem above the circuit spins and bobs continuously. Background music plays throughout. The HUD displays speed, score, lap time, and the active camera mode, all rendered with a drop shadow effect.

Top-Down Scene Layout



Figure 1: Annotated top-down scene layout of Drive Circuit.

2. Controls

Key	Action
W	Accelerate — increases player speed along the spline
S	Brake / decelerate — reduces speed; stops at zero
F	Free camera — WASD movement and mouse look (template camera)
1	First person camera — eye above car looking forward along road
2	Third person camera — behind and above car, road visible ahead
3	Top view camera — overhead, follows player, tangent as up
L	Toggle Night mode on / off
K	Toggle Storm mode (enables night on; returns to day on off)
ESC	Quit the application

3. Route and Camera

3a. The Spline Route

I wrote a `CCatmullRom` class that takes a list of user-defined control points and builds a closed Catmull-Rom spline from them. The standard Catmull-Rom basis with tension 0.5 gives C1 continuity — tangents match at every control point junction, so the road flows smoothly through corners without kinks.

I sample the spline at 100 steps per segment. With 16 control points that give 1,600 centerline positions. I also build a cumulative arc-length table at the same time, which I use to map a normalized parameter t in $[0,1]$ uniformly to positions around the full circuit. This means the player moves at a consistent world-space speed regardless of how tight or wide a section of the track is.

The `CTrack` class takes the spline and generates the road ribbon as a `GL_TRIANGLE_STRIP`. For each centerline sample, I compute the left and right edge positions by offsetting along the horizontal normal, which I get from `cross(worldUp, tangent)`. I zero out any Y component and renormalize to keep the road flat. All road vertices sit at $Y = 0.2$. I use `glPolygonOffset(-4, -4)` and `GL_LEQUAL` depth testing when drawing the road, so it always renders cleanly on top of the grass without Z-fighting.

The road edges are demarcated in three layers. First, 40 CDiamond primitives are placed at $t = k/40$ alternating left and right, sitting just outside the road edge. They glow yellow in day mode and orange at night, and all 40 spin and bob independently with per-marker phase offsets so they create a ripple wave effect as you drive past. Second, a white centerline strip is generated as an inline VAO and rendered as a thin GL_TRIANGLE_STRIP on top of the road. Third, two road edge glow strips run the full length of the circuit and light up orange in night mode.

3b. Camera Modes

I kept the template free camera (F) unchanged for debugging. The three additional modes all compute the camera position from the TNB frame at the player's current spline position t :

- **First Person (1):** Eye placed 14 units above the player position and 5 units ahead along the tangent, looking 30 units further forward. Gives a clear driver's view of the road.
- **Third Person (2):** Camera sits 50 units behind and 20 units above the player, looking at the player's position offset 4 units upward. The car appears in the lower portion of the screen with the road curving ahead.
- **Top View (3):** 250 units directly above the player, looking straight down. I use the tangent as the world-up vector so the road always reads left-to-right in the direction of travel.

4. Basic Objects, Meshes, and Lighting

4a. Basic Objects

I created two custom geometric primitives from scratch:

- **CDiamond** — An octagonal cut diamond shape built from 16 triangular faces (8 crown, 8 pavilion). It uses 48 interleaved VBO vertices with a stride of 32 bytes (position 12B + UV 8B + normal 12B) and is drawn as two GL_TRIANGLES calls, one per texture half. Winding order and face normals were computed by hand in the interim coursework. I reuse this shape as the floating central gem and as the 40 animated lane markers around the track.
- **CTrack** — The spline-generated road ribbon described in Section 3a. Same interleaved stride-32 layout as CDiamond, rendered as a GL_TRIANGLE_STRIP. Normals are all (0,1,0) since the road is flat, and UV coordinates tile along the road length.

4b. Meshes

I loaded four external OBJ models using the Assimp-based COpenAssetImportMesh loader. The template horse and barrel meshes do not count toward this total.

Mesh	File	Scale	Instances	Position in scene
Player car	Car.obj	8×	1	On spline at player t, rotated to face tangent direction
Tree	Tree.obj	10×	8	Fixed positions at ±550 X / ±520 Z, outside the track
Statue	statue.obj	0.25×	4	±520 units on the four cardinal axes
Stone Arch	Stone.obj	5×	4	±600 units on the four cardinal axes

For the player car I build the rotation matrix manually from the TNB frame: $\text{carRight} = \text{cross}(\text{worldUp}, T)$, $Y = \text{worldUp}$, $Z = T$. This is combined with a translation and uniform scale matrix into a single model-view matrix, bypassing the MatrixStack to avoid the unsupported *= operator.

The five barrel pickups (template mesh) are placed along the spline at $t = 0.1, 0.3, 0.5, 0.7, 0.9$ as collectibles. The horse (also template) is positioned off-route at (350, 0, 0).

4c. Lighting

The scene uses per-vertex Phong shading implemented in mainShader.vert. Three lighting configurations are available:

- **Day mode:** A directional sunlight source at world position (-100, 200, -100). $L_a = 0.3$, $L_d = 1.0$, $L_s = 1.0$. No spotlights, no fog.
- **Night mode (L):** Sunlight diffuse and specular set to zero; ambient drops to (0.25, 0.25, 0.30). Two spotlights activate. Spotlight 1 is white ($L_d = 1.0$), 12 units above the player, aimed forward along the tangent with slight downward tilt, 55° cone — acts as the car headlight. Spotlight 2 is orange ($L_d = (1.0, 0.5, 0.1)$), fixed at the far side of the circuit 60 units overhead with a 60° cone. Exponential fog (density 0.0008, deep navy) adds atmosphere.
- **Storm mode (K):** Extends night mode. Fog density increases to 0.006 with stormy blue-grey colour. Spotlight 1 cone narrows to 30°, simulating rain scattering the beam. Rain streaks and lightning activate in the fragment shader (see Section 5c).

5. HUD, Gameplay, and Advanced Rendering

5a. Heads-Up Display

The HUD uses CFreeTypeFont with an orthographic projection matrix, drawn after the 3D scene with depth testing disabled. It shows:

- FPS — top left, white
- Speed in km/h — yellow, scaled from normalized player speed
- Score — number of barrel pickups collected this session, cyan
- Lap time in seconds and centiseconds — orange, resets on each lap
- Active camera mode — top right, white
- NIGHT MODE indicator in purple or STORM MODE in blue when those modes are active
- Controls hint bar at the bottom of the screen

For the text rendering I implemented a drop shadow technique. Every element is drawn twice: first a dark semi-transparent copy (color 0,0,0 at 65% opacity) offset by +2 pixels right and -2 pixels down, then the full-color text at the correct position. This gives the HUD a clean 2.5D depth look that stays readable whether the background is bright grass, dark night, or a white lightning flash.

5b. Gameplay

The player uses W to accelerate and S to brake. Speed accumulates up to a cap of 0.08 t/s, which displays as roughly 120 km/h. Each frame the player position t advances by $\text{speed} \times dt$ and wraps at 1.0, triggering a lap reset.

For collision detection I check `glm::length(playerPos - barrelPos)` against a 12-unit threshold for each of the five barrels every frame. When a barrel is collected the score increments and the barrel is marked so it does not score again until the next lap. Collected flags reset when t wraps.

All 40 diamond lane markers and the central gem are animated in real time. Each lane marker spins at $90^\circ/\text{s}$ with a per-marker spin offset ($mk \times 9^\circ$) and bobs ± 0.5 units on a sine wave with a per-marker phase offset ($mk \times 0.5$ radians). This creates a visible ripple wave along both sides of the track as you drive past. The central gem spins at $60^\circ/\text{s}$ and bobs ± 2 units, making it the most prominent animated object in the scene.

5c. Advanced Rendering Techniques

Technique 1 — Exponential Fog (shader): In `mainShader.vert` I compute the eye-space distance of each vertex as `length(eyePos.xyz)` and calculate `fogFactor = exp(-fogDensity × distance)`, clamped to `[0,1]`. In `mainShader.frag` the lit fragment color is blended toward a fog color using `mix(fogColour, litColour, fogFactor)`. The fog density and color change between all three modes: zero in day, 0.0008 with deep navy in night, 0.006 with stormy blue-grey in storm. This is a full shader implementation — no CPU-side computation is involved.

Technique 2 — Multi-Texturing (shader): The road surface blends two textures in the fragment shader. Texture unit 0 carries the base road texture (`dirtpile01.jpg`), bound by `CTrack::Render()`. Texture unit 1 carries a detail overlay (`Tile41a.jpg`) loaded separately into `m_detailTexture` and bound manually to `GL_TEXTURE1` before the road draw call. In the fragment shader, when `bMultiTex` is true, I sample both textures, tile the detail 4× more finely, and blend with `mix(t0, t1, 0.35)` to add micro-detail to the road surface.

Technique 3 — Storm Rain and Lightning (shader): When `bStormMode` is true in the fragment shader, three overlapping rain streak layers are computed using `fract()` on combinations of `vTexCoord` and `uTime` at different frequencies and scroll speeds, producing animated diagonal streaks across the scene. For lightning I multiply three sine waves at different frequencies — $\sin(t \times 1.3) \times \sin(t \times 7.7) \times \sin(t \times 13.1)$ — and raise the product to the power of 3. This produces sharp irregular spikes that add a white-blue brightness flash to the whole scene unpredictably. The entire effect is fragment-shader only, driven by the `uTime` uniform passed from the game clock.

6. Scene Assets

Asset	Source	Downloaded	License
Car.obj + Car.mtl	https://free3d.com/3d-model/low-poly-car-40967.html	May 2026	Free personal use
Tree.obj + Tree.mtl	https://free3d.com/3d-model/tree-74556.html	May 2026	Free personal use
statue.obj + Statue.mtl	https://free3d.com/3d-model/statue-v1--541832.html	May 2026	Free personal use
Stone.obj + Stone.mtl	https://free3d.com/3d-model/giant-stone-765391.html	May 2026	Free personal use
DST-Garote.mp3	http://www.nosoapradio.us	Template	Free with attribution
grassfloor01.jpg, dirtpile01.jpg, Tile41a.jpg	OpenGL Template — Dr E. Edwards	Template	Academic use
Horse.obj, Barrel02.obj	OpenGL Template — Dr E. Edwards	Template	Academic use
OpenGL Template v6.1a	City, University of London (Moodle)	Feb 2026	Academic use

7. Discussion

Strengths

- The Catmull-Rom spline with arc-length parameterization works well. The player moves at a consistent speed through both tight hairpins and long straights, cameras attach

cleanly to the TNB frame, and the road geometry generates correctly at any circuit layout just by adjusting the control points.

- The three visual modes — Day, Night, Storm — feel meaningfully different. The storm mode in particular, with its animated rain and unpredictable full-scene lightning, gives the game a distinctive atmosphere that is not easy to dismiss as a simple color change.
- The animated lane markers are the part I am most pleased with. The ripple wave effect along both sides of the track as you drive past gives the circuit genuine personality, and reusing CDiamond from the interim coursework for both the central gem and the markers was an efficient decision.
- The drop shadow HUD stays readable in all three modes, including during bright lightning flashes, which was a real concern during development.

Weaknesses

- The car is a single rigid mesh. The wheels do not turn. A proper implementation would require separate wheel meshes that rotate on their own axis relative to the body, which would need a hierarchical matrix stack or manual sub-transforms per wheel.
- Collision detection is a simple point-to-point distance check. It works well enough for barrels but there is no road boundary detection — driving off the track has no gameplay consequence.
- The rain effect is a screen-space UV trick. It does not respect depth or scene geometry, so rain looks equally thick right next to the car and at the horizon.

What Would Expand This into a Complete Game

- Multiple AI-controlled opponents following the same spline at different speeds and positions would turn the time-trial into a real race. The spline infrastructure already supports this.
- A proper physics model with tyre grip, weight transfer, and braking distance would make cornering skill matter.
- Shadow mapping from the spotlights would add ground shadows in night mode, dramatically improving atmosphere.
- A geometry-shader particle system for rain and tyre-dust effects would replace the UV rain trick with physically positioned particles that respect depth.

Appendix: Use of Generative AI

AI assistance (Claude Sonnet, Anthropic) was used solely to format and structure the layout of this Word document.